OnboardAI Project Guide (Final Review)

This guide explains the whole project twice. Part A uses ordinary language and assumes no background in computing. Part B covers the same ground with the technical detail an examiner will expect.

Read Part A first, even if you are technical: it is the story the team tells. Read Part B to be able to answer "how" questions. The mid term version of this guide is kept as `PROJECT_GUIDE_MIDTERM.md`.

Team: Kritika Sah (Team Lead), Rohith K Nair (Developer, all application code), Prisha Saini (Developer, evaluation and tests), Pasham Srikar (Developer, risk model and data pipeline).

Course: Introduction to AI and Machine Learning, Department of Artificial Intelligence and Data Science, Amrita Vishwa Vidyapeetham, Delhi NCR. Subject coordinator: Dr. Sakshi Ahuja.

Part A: The Project in Plain Language

A.1 What problem are we solving

When a company hires somebody, a surprising amount of paperwork follows. The new employee has to send in a copy of their passport or identity card, a signed offer letter, bank details so they can be paid, and depending on the job, things like a safety certificate or a professional licence. Somebody in the human resources (HR) team has to look at each of these and decide whether it is acceptable.

At the same time the new employee has questions. How much holiday do I get? When does my training start? Who do I talk to about my laptop? These are not difficult questions, but they have to be answered, and the answers differ from company to company.

All of this is done by hand in most organisations. It is slow, repetitive, and easy for somebody to fall through the gap. A new employee left waiting for a week forms a poor opinion of the company, and replacing somebody who leaves in their first year costs several months of their salary. So this is an expensive problem, not a small annoyance.

A.2 What our system does

OnboardAI is a website with two kinds of user: new employees and HR staff. It helps with three things.

1. Checking documents. When an employee uploads a document, the system reads it, works out what kind of document it is, pulls out names, dates and signatures, and decides one of three things: approve it, reject it with a reason, or send it to a person in HR. If a safety certificate expired two years ago, it rejects it and says so. If it is not sure, it never guesses: it asks a human.

2. Answering questions. Employees chat with an assistant. Behind the chat there are six specialist "agents", each responsible for one area (documents, IT access, policies, and so on). A small trained model decides which specialist should handle each message. Policy questions are answered only from the company's own handbook, with the section quoted. If the handbook does not cover the question, the assistant says it does not know and offers to pass it to HR. It is not allowed to make up an answer, because a confident wrong answer about leave or pay causes real harm.

3. Spotting people who are falling behind. The system looks at each new employee's progress, how long since they last did anything, rejected documents, and how their messages sound, and estimates how likely they are to get stuck. It also explains why, for example "48 per cent, mainly because no tasks are complete and one document was rejected".

Two companies use the system in our demonstration: Nimbus Labs, a fintech start-up, and Veltrix Corp, a manufacturer. They have different policies, and neither can ever see the other's handbook.

A.3 Where we are now

Everything is built and measured.

- **Mid term (complete):** we built the three datasets, analysed them, cleaned and prepared them, split them into training and testing portions, and wrote automated checks.
- **Final review (complete):** four trained models, the document checking pipeline, the chat assistant with six agents, the risk predictor, a website that ties them together, and an independent evaluation written by hand after training.

The whole system runs on one laptop. The language model behind the chat (Qwen 2.5, 7 billion parameters) runs locally through a program called Ollama, so there is no cost, no internet dependency, and no employee message leaves the machine.

A.4 Where our data came from, and why that matters

All three datasets are generated by our own programs rather than collected. Real identity documents and bank details cannot lawfully be gathered for a student project, and real onboarding outcomes do not exist because the system has never been deployed.

We made the generated data deliberately difficult: documents are damaged like bad phone scans, some messages are genuinely ambiguous, and the outcome of each simulated employee depends partly on something the model can never see. Even so, a model trained on generated data can only prove it learned our generator. That is why, for the final review, we wrote a separate set of test cases by hand, in natural wording, and measured the system on those. Section A.5 explains what that showed.

A.5 What we found

1. Natural wording is harder than our generated data. The model that reads chat messages scores 94 per cent on generated test messages but 73 per cent on messages we wrote by hand. It learned the vocabulary of our templates. This is the most important honest finding of the project, and we show it openly. The system copes because a wrongly routed message can be passed to the right specialist, and an uncertain message goes to the general assistant instead of being forced into a specialist.

2. Document checking is strong and, above all, safe. On 16 hand-written documents covering every kind of problem (unsigned, expired, wrong person, wrong document, damaged scan, photo) the system gave the right verdict for all 16 and approved no bad document.

3. Testing on hand-written cases found real bugs. An empty "Signature: ____" line counted as signed. A photo was rejected as unreadable just because no text-reading (OCR) software was installed. Certificates worded "This certifies that..." never matched the employee's name. Policy search treated "gift" and "gifts" as different words. An agent that received a transfer tried to transfer again instead of answering. All of these are fixed and have tests that stop them coming back.

4. Our own data had a leak, and we measured it rather than assumed. At mid term we found 70 duplicate messages in both the training and test portions. We tested whether this inflated our score. It did not: the duplicated messages were actually the hardest ones.

A.6 Who did what

Four of us worked on this. For the final review the team is one Team Lead and three Developers; the separate Tester and QA roles used at mid term were removed.

- **Kritika Sah (Team Lead):** defined the problem, set the objectives and scope, planned the work and leads the presentation.
- **Rohith K Nair (Developer):** wrote the application code: the document checking pipeline, the chat assistant and its agents, policy search, and the website and server.

- **Prisha Saini (Developer):** owns evaluation: the hand-written test cases, the results shown in the Evaluation Lab, and the automated test suite (62 checks).
- **Pasham Srikar (Developer):** owns the risk prediction model and the data pipeline: the three data generators, the preparation of the data and the training/testing split.

Part B: The Project in Technical Terms

B.1 System overview

OnboardAI is a full stack application with the machine learning and natural language processing layer in the request path, not attached as a separate demonstration.

- **Front end:** React 19, Vite, Tailwind CSS.
- **Back end:** FastAPI, SQLAlchemy over SQLite.
- **Machine learning:** scikit-learn and NumPy.
- **Language model:** Qwen 2.5 7B through Ollama, called through LangChain. Google Gemini and Groq are available behind a provider switch in `backend/.env`.
- **Agents:** LangChain `create_agent` with LangGraph in-memory conversation history.

Problem	Data	Target	Samples	Model
Document type	Text	9 classes	4,950	Linear SVM, calibrated
Message intent	Text	8 classes	3,360	Logistic regression
Message sentiment	Text	3 classes	3,360	Logistic regression
Onboarding stall	12 numeric features	Binary	6,000	Logistic regression

B.2 Data generation methodology

Document corpus. Twenty one templates across nine classes, with randomised names, dates, identifiers and organisations. Degradation is applied in three tiers (about 40, 38 and 22 per cent). The severe tier strips the header, cuts the document to a fragment, injects 10 to 20 per cent character noise using OCR confusion pairs, and appends sentences from other classes. A first version with no degradation scored 1.000 with zero variance; that was treated as a broken benchmark and the generator was rebuilt.

Message corpus. Forty five templates across eight intents, each message carrying two independent labels (intent and sentiment). Irreducible error is introduced through uninformative messages ("any update?") whose label cannot be recovered from the text, and blended messages spanning two intents.

Journey dataset. A latent variable process. Three hidden traits per simulated employee (engagement, friction, support) drive the 12 observable features through noisy functions, for example:

```
days_since_last_activity ~ Exponential(exp(0.75 - 0.65 * engagement + 0.30 * friction))
avg_sentiment            = tanh(0.55 * engagement - 0.85 * friction + 0.45 * support + e)
```

The outcome is a Bernoulli draw on a logit of the traits plus an explicitly unobserved term, which puts a ceiling on achievable accuracy by construction.

B.3 Preprocessing

Text. Lowercasing and accent stripping, then two TF-IDF feature sets joined together: words (one and two word sequences) and characters (three to five character sequences within words). Sublinear term frequency; minimum document frequency 2. The character features are the deliberate choice: scan noise destroys whole words but most three to five letter fragments survive.

Tabular. Standard scaling inside a scikit-learn `Pipeline`, so it is refitted on each training fold and never sees the test fold.

Not performed. Missing value imputation (there are no missing values, verified by test TC-02) and outlier removal (the outliers are long inactivity and waiting times, which are exactly the behaviour the risk model must detect).

B.4 Exploratory analysis findings

1. Document and intent classes are balanced (ratio 1.00), so plain accuracy is a fair headline. Sentiment is imbalanced by design, so balanced class weights are used.
2. `stage`, `progress` and `pct_tasks_complete` correlate above 0.90 with each other (0.97 for the strongest pair), which explains why removing any one of them barely changes results.
3. No single journey feature correlates with the outcome above 0.45; the strongest is `avg_sentiment` at about -0.45.
4. Sentiment separates the outcome classes in the expected direction, with heavy overlap: a contributing signal, not a decisive one.

B.5 Splitting and leakage audit

All datasets use a 75/25 stratified split with `random_state=42`. Class proportions match the full data to within 0.0005. The vectoriser and scaler sit inside the pipeline, so cross validation cannot leak test information into them.

The audit found 200 exact duplicate messages, 70 of which appear in both folds (8.3 per cent of that test set). We tested whether this inflated the score:

Partition	Rows	Sentiment accuracy
Also appears in training	76	0.9868
Not in training	764	0.9974

The leaked rows score lower, so the leak did not inflate results; the duplicates are the short, ambiguous, hardest messages. It remains a documented defect, bounded by two tests.

B.6 Model results on held-out generated data

These are the numbers on the 25 per cent test split of our generated data, with 5-fold cross validation.

Document classifier. Calibrated linear SVM chosen over logistic regression (0.9863 against 0.9838 accuracy, and faster). Test accuracy 0.9863, macro F1 0.9863, cross validated macro F1 0.9859 ± 0.0040. By scan quality: clean 1.000, degraded 1.000, severe 0.937.

Intent classifier. Logistic regression. Accuracy 0.9369, macro F1 0.9368, cross validated 0.9331 ± 0.0037. Weakest classes are document help and status check (F1 about 0.91).

Sentiment classifier. Logistic regression with balanced class weights. Accuracy 0.9964. This is a ceiling set by how separable our templates are, not real-world ability; on hand-written messages it scores 0.667.

Risk model. Logistic regression chosen over gradient boosting (ROC-AUC 0.7948 against 0.7773, and better calibrated). Cross validated ROC-AUC 0.8070 ± 0.0019. Calibration is good: when it predicts 0.88 the observed stall rate is 0.88. The most important feature is average message sentiment.

B.7 How the components work

Document pipeline (`backend/ml/pipeline.py`). Five steps: extract text, classify the type, extract fields, run validation checks, decide.

1. Text comes from the PDF text layer; scans and photos fall back to OCR if installed.

2. The classifier gives a type and a confidence.
3. Rules extract names, dates (labelled as issue, expiry or birth by the nearest label), identifiers and the signature state (signed, blank field, or absent).
4. Checks: readable, type matches what was requested, not expired (warning if within 30 days), name matches the employee, signed where the type requires it.
5. Decision: any blocking failure rejects; a warning, confidence below 0.80, or unreadable scan sends it to HR; otherwise approved.

Chat assistant (`backend/agents`, `backend/routes/chat.py`). For every message:

1. The intent and sentiment classifiers read the message (about 10 ms, no language model).
2. Personal data (email, phone, Aadhaar, PAN, passport, IFSC, account and card numbers) is replaced with placeholders before anything reaches the language model.
3. If intent confidence is at least 0.45, the message goes to the matching specialist; otherwise to the general Onboarding Agent.
4. The specialist runs with only its own tools. Each tool is locked to the signed-in employee, so no prompt can reach another person's record.
5. If the specialist decides the message is outside its remit, it can transfer once. The receiving specialist cannot transfer again; it must answer.
6. A message classified as an escalation, or with sentiment at or below -0.60, is flagged to HR.

The six specialists and their tools:

Specialist	Tools
Onboarding Agent (default)	get status, list tasks, complete a task
Document Agent	list documents and rejection reasons, list tasks
IT Provisioning Agent	raise an IT request, list tasks
Knowledge Transfer Agent	list knowledge-transfer sessions
Policy Agent	search the company handbook
Escalation Agent	escalate to HR, get status

Policy search (`backend/ml/rag.py`). The handbook (8 documents, 30 sections) is split by section. Each search scores sections half by exact word overlap (TF-IDF) and half by meaning (LSA, a compressed version of the same matrix that links words used in similar contexts). Plurals are folded so "gift" matches "Gifts". Only the employee's own company's documents, and only sections their department may see, are searched. A best score below 0.42 means "no real answer": the agent is told to decline, and the question is logged as a documentation gap that HR sees on its dashboard.

Risk scoring (`backend/ml/risk_model.py`). Logistic regression over 12 standardised features. Each feature's contribution to the log odds is exact (for a linear model these equal SHAP values), so every score comes with its main drivers.

B.8 Final review evaluation

Every case was written by hand after training, in wording the generators never used. The cases are in `backend/evaluation`, run live in the app under HR login, Evaluation Lab, and offline with `python -m evaluation.report`.

What was tested	Cases	Result
Intent of a hand-written message	48	72.9 per cent (93.7 on generated data)
Sentiment of a hand-written message	48	66.7 per cent

What was tested	Cases	Result
Policy search: right section ranked first / in top 3	12	83.3 / 91.7 per cent
Policy search: finds nothing when the handbook has no answer	2	2 of 2
Policy search: never shows the other company's handbook	2	2 of 2
Full answers: correct fact	12	11 of 12
Full answers: cites the handbook section	12	9 of 12
Full answers: declines when it should	4	3 of 4
Document verdicts correct	16	16 of 16
Bad documents wrongly approved	16	0

Average full answer time is about 30 seconds on this laptop; classification and document checks take milliseconds.

B.9 Defects found and their resolution

Ref	Defect	Found	Resolution
1	Date role used whole-line keyword search, marking a valid passport expired	Mid term	Nearest preceding label, test TC-18
2	Certificate number pattern matched the word CERTIFICATE	Mid term	Required qualifier token, test TC-19
3	Two risk features fixed at inference but varied in training	Mid term	Features removed, model retrained
4	Positive sentiment labels with no positive wording	Mid term	Marker guaranteed in generator
5	Duplicate messages straddle the split	Mid term	Measured, bounded by tests, fix deferred
6	Empty signature line read as signed	Final	Signature state: signed, blank or absent; blank goes to HR
7	Photo auto-rejected when no OCR is installed	Final	Sent to HR with a clear reason
8	"This certifies that" names not found	Final	Pattern added
9	"gift" did not match "Gifts" in policy search	Final	Plural folding; pass rate 75 to 87.5 per cent
10	Agent receiving a transfer tried to transfer again	Final	Receiving agent has no transfer tool
11	Employee saw "awaiting verification" after an automatic verdict	Final	Verdict shown immediately

Every final review fix has a regression test in `backend/tests/test_final_review_fixes.py`. The suite is now 62 checks, all passing, in about 20 seconds, with no network access.

B.10 Limitations

1. All training data is generated. The hand-written evaluation shows the cost: intent accuracy falls from 94 to 73 per cent on natural wording.
2. The same team wrote the hand-written cases and the fixes. A test set written by someone outside the team would be a stronger check.
3. Sentiment accuracy on generated data (0.996) is a template ceiling, not real ability.
4. Generated documents are short and simpler than real scans; there is no OCR engine installed, so photos go to HR.
5. The 7B local model sometimes answers without citing the section, and full answers take about 30 seconds.
6. The application has no real authentication (demo logins only) and is not under version control.

B.11 Future work

1. Train the message classifiers on real or much more varied messages; this is the largest gain available.
2. Replace the document corpus with a public one such as RVL-CDIP and install OCR.
3. Deduplicate messages before splitting.
4. Use HR's overrule decisions, which are already logged, as new training labels.
5. Add real authentication and move the project into version control.